Laravel After Deploy

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

Laravel After Deploy

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

First published: August 2026

Written with	Markdown, part of it dictated with Hex, an open-source, on-device voice-transcription tool, and the rest typed.
Produced with	Ibis Next, using mPDF for PDF generation and CommonMark for HTML rendering.
Trim size	7.44in x 9.69in (Crown Quarto)
Typefaces	Linux Libertine (body text), Times (headings), and 0xProto (code, SIL Open Font License 1.1).
Code width	66 characters per line, so a listing never wraps awkwardly on the page.

To my family, for making every effort worthwhile.

Preface

Shipping is the easy part. The hard part starts after deploy. A flash sale or a viral post turns usual traffic into a spike. Bots hammer your login endpoint. A dependency you don't control has a bad afternoon. The same webhook arrives twice. The on-call phone rings at 3 a.m. and the answer isn't in the framework docs. This book is about that layer: the decisions that keep a real system fast, correct, and operable once it's already in production.

Every code example is Laravel and PHP. That is a practical choice. One stack, end to end, so each failure can be shown against real framework seams: Eloquent, queues, Horizon, Sanctum, the container, the scheduler. The problems themselves are not Laravel-only. Idempotency, backpressure, observability, failure isolation, and schema changes under load show up in any language. If you can read PHP, you can carry the patterns home even when your implementation looks different.

Who this book is for

This book is written first for a mid-to-senior Laravel engineer who already knows how to build an application and is now being asked to keep one alive: architecture decisions, performance incidents, migrations that have to ship without a maintenance window. It assumes routing, Eloquent, queues, and testing, and does not re-teach the framework. The pages are about the layer on top: structuring a codebase once more than one team touches it, keeping it fast when traffic outgrows one comfortable database, correct when the same request can arrive twice, and available when a dependency goes down; then running all of that at 3 a.m. on a Tuesday without turning it into a story you still tell a year later.

It is also for backend and platform engineers on other stacks who want a production playbook shown in one mature PHP framework. You do not need to ship Laravel day to day. You do need enough PHP to follow the listings, and you will see Laravel names in the text: Horizon, Sanctum, Eloquent, and the rest. It is not a polyglot cookbook or a framework comparison.

If you're looking for a Laravel tutorial (how to define a route, how to write a migration, how Eloquent relationships work), this isn't that book, and there are excellent ones already. If you're staring at a monorepo, a queue backlog, a balance that doesn't add up, or a pager you don't trust yet, keep reading.

How to read it

The book has eight parts, ordered by dependency rather than importance. Reading front to back works, but it is not the only sane path. Part 1 (Foundations) and Part 2 (Performance Engineering) build vocabulary later parts assume: "shared kernel," "N+1," "backfill," and so on. Part 5 (Observability & Monitoring) is worth reading earlier than its page number suggests, because "correlation ID," "production dashboard," "SLI/SLO/SLA," and "burn rate" get reused heavily in Parts 6 and 7. With those vocabulary caveats, Parts 3 through 7 are mostly skippable during a live fire. If you picked this up because of an incident with queues, database failover, or CI, jump straight to the relevant chapter. Chapter 46, the capstone, assumes you have read or at least skimmed everything before it. It is one worked incident that touches nearly every decision in the book, and it is meant to be read last.

Each chapter follows the same shape. It opens with a concrete production failure, drawn from more than fifteen years of real incidents and anonymized. It then works through the pattern that would have prevented or fixed it, and closes with a short "Chapter recap" you can scan later without rereading the whole chapter. Not every incident started in a Laravel project. Over those years I have worked across many languages and frameworks, and where the original system was not Laravel, I translated the failure into a Laravel setting so it can be demonstrated in the stack this book uses.

About the examples

Every incident described in this book actually happened in real production systems. The failure modes, the trade-offs, and the fixes come from systems I helped build and then had to keep running, not from thought experiments. None of the identifying details are real. Company names, product names, team names, exact numbers, and anything else that could point back to a specific employer or client has been changed, generalized, or recombined. The named fictional businesses that recur throughout the book are stand-ins, not case studies of any single company:

Name	Domain	Typical chapters
Adda	Social platform; abuse and rate limiting	16
Chanda	Subscription billing, queues, IaC	13, 37
Dokan	Multi-tenant SaaS, release checklist	10, 36
Drishti	Livestreaming and short-video; real-time, cost/capacity	11, 15, 25, 38-39, 44
HaatBazaar	Marketplace (search, inventory); capstone synthesis	18, 21, 46
Hishab	Wallets, ledger, payouts, observability	19-20, 22-23, 28, 31-32, 40, 42

Name	Domain	Typical chapters
Jatri	Ride-dispatch; runbooks and on-call	43
Karbar	Order-processing platform	1-3, 8-9, 17, 26, 29, 34-35, 41
PadmaPay	Fictional payment provider (vendor dependency)	5-6, 20, 30, 32, 46

Those chapter lists are the main appearances, not an exhaustive index. Several of these names show up elsewhere when the same failure mode needs a familiar face. PadmaPay is the recurring third-party payment vendor the cast integrates with, never a first-party system under discussion.

Nothing here discloses a trade secret, a proprietary architecture, or confidential information belonging to any organization I've worked with. The anonymization is there to protect that while keeping the underlying lesson intact. The architecture decisions and opinions in these pages are my own.

Reading paths for live incidents

If something is on fire, these paths get you to the relevant chapters without reading front to back. Skim the glossary first if a term is unfamiliar.

- **Money, balance, or payout looks wrong** → 19 (idempotency, state machines) → 20 (webhooks) → 22 (ledger and fund holds). Add 23 only if you need rejected-attempt history and audit reconstruction, not just successful movements. Overselling / inventory races → 21.
- **Deploy went green but nothing changed** → 35 (containers) → 36 (release checklist: workers, scheduler, `/ready`) → 37 (IaC roles).
- **Multi-app monorepo or auth pain across services** → 3 (monorepo) → 34 (CI path filters) → 41 (Sanctum vs JWT).
- **On-call can't see what's happening** → 31 (logs/metrics/traces) → 32 (production dashboard) → 33 (SLOs and alerting) → 43 (runbooks).

A note on unfamiliar terms

Abbreviations are spelled out the first time a chapter uses them. If you're dropping into a chapter out of order, Appendix A (Glossary) collects the recurring ones in one place: SLO, SLA, SLI, MTTR, RPO/RTO, ADR, and the rest. Each carries a pointer back to the chapter that covers it properly.

Chapter 9 - Safe Schema Evolution at Scale

The migration that only failed in production

Renaming a column is the kind of migration nobody argues about in review. At Karbar, a fictional order-processing platform, a cleanup task turns `orders.buyer_email` into `orders.customer_email` so the column matches the naming used everywhere else in the domain. Against staging's ten-thousand-row table it runs in forty milliseconds. Against production's two hundred million rows, the `ALTER TABLE` takes an exclusive lock for twenty-five minutes. Every write to `orders` blocks behind it, the API starts returning timeouts, the on-call engineer gets paged, and the migration has to be killed mid-run, leaving the schema in an ambiguous state.

Nothing in the migration was syntactically wrong. It was tested at the wrong scale, which is Chapter 8's failure mode pointed at schema changes instead of queries. Four patterns keep a schema change safe whatever the table size: non-blocking DDL, a backward-compatible rename, batched backfills, and the one teams forget most often, tracking who else reads your schema before you decide a "safe" change is safe for everyone.

Why "fast in staging" doesn't transfer

What a schema change costs depends on data volume, the database engine, and the specific operation, not on how simple the statement looks:

- Adding a nullable column with no default is metadata-only and near-

instant on both MySQL (InnoDB, modern versions) and PostgreSQL, regardless of table size.

- Adding a column with a default value, or adding a `NOT NULL` constraint to an existing column, historically required rewriting every row to populate or validate the new value, a cost proportional to table size. Recent versions of both engines have narrowed this. PostgreSQL avoids a full rewrite for constant defaults since version 11, and MySQL's behavior depends on the specific `ALGORITHM` used. Verify the operation against the engine version you actually run, on a copy of production-sized data, before trusting it in staging.
- Renaming a column, dropping a column, or changing a column's type can each range from instant metadata changes to full table rewrites, again depending on engine and version.

One rule survives all that engine-specific detail: never trust a migration's timing from a small staging database. Run it against a production-sized copy first, a recent snapshot restored somewhere disposable, and know before you ship whether the specific operation takes a blocking lock.

Adding a `NOT NULL` constraint to `orders.customer_email` on PostgreSQL as a single step makes the database scan and validate every one of two hundred million rows while holding a lock that blocks writes for the duration:

```
/*
 * Locks orders for the full validation scan - the whole table,
 * all at once.
 */
Schema::table('orders', function (Blueprint $table) {
    $table->string('customer_email')->nullable(false)->change();
});
```

Split it into three steps: add the constraint without enforcing it, validate it separately, then make the column formally `NOT NULL`. One long exclusive lock becomes a near-instant metadata change plus a scan that takes a brief lock only at the very end:

```
-- Step 1: instant - records the constraint without checking
existing rows.
ALTER TABLE orders ADD CONSTRAINT customer_email_not_null
    CHECK (customer_email IS NOT NULL) NOT VALID;

-- Step 2: scans the table, but only takes a brief lock to flip
the
-- constraint's state once the scan finds no violations - not for
the
-- duration of the scan itself.
ALTER TABLE orders VALIDATE CONSTRAINT customer_email_not_null;

-- Step 3: after the validated check proves there are no nulls,
PostgreSQL
-- can set the column attribute without repeating the full-table
scan.
ALTER TABLE orders
    ALTER COLUMN customer_email SET NOT NULL;
```

Run as separate deploys, the first one is safe to ship immediately; validation and the final `SET NOT NULL` can run once the backfill from the next section has finished, with no code depending on their timing.

Concurrent and non-blocking index creation

Adding an index to a large table is one of the most common migrations to get wrong. A plain `CREATE INDEX` takes a lock that blocks writes for as long as the build takes, and on a two-hundred-million-row table that can be tens of minutes.

PostgreSQL's answer is `CREATE INDEX CONCURRENTLY`. It builds the index

without holding a write lock. You pay for that with a slower two-pass build and one real failure mode: an interrupted build leaves behind an invalid index that has to be dropped and retried rather than resumed.

```
CREATE INDEX CONCURRENTLY idx_orders_customer_created
    ON orders (customer_id, created_at);
```

MySQL's InnoDB engine supports online DDL for many index operations through `ALGORITHM=INPLACE, LOCK=NONE`, which allows concurrent reads and writes during the build. Not every DDL operation supports every algorithm, so check the one you need against the engine version you're running instead of assuming.

```
ALTER TABLE orders
    ADD INDEX idx_orders_customer_created (customer_id,
created_at),
    ALGORITHM=INPLACE, LOCK=NONE;
```

Laravel's migration files don't manage this distinction for you. A `Schema::table()` migration using `$table->index()` issues a plain `CREATE INDEX` or `ADD INDEX` unless you write the raw SQL or pass a database-specific driver option to request the non-blocking variant. On a large table, make that call explicitly instead of inheriting the default:

```
public function up(): void
{
    /*
     * Schema::table()->index() takes a blocking lock for the
     * build - fine on a small table, a production incident on a
     * two-hundred-million-row one.
     */
    DB::statement(
        'CREATE INDEX CONCURRENTLY idx_orders_customer_created'
        .' ON orders (customer_id, created_at)'
    );
}
```

`CREATE INDEX CONCURRENTLY` can't run inside the transaction Laravel wraps each migration in by default, so the migration class also needs `public $withinTransaction = false;`. Forgetting that is the most common way this pattern fails silently in a migration file while working fine when you run the statement by hand. An interrupted concurrent build leaves an unusable index behind rather than rolling back, so always check for one before retrying:

```
-- Finds indexes left in an invalid state by an interrupted
CONCURRENTLY build.
SELECT indexrelid::regclass AS index_name
FROM pg_index
WHERE indisvalid = false;

-- Safe to drop and retry - an invalid index is never used by the
planner.
DROP INDEX CONCURRENTLY idx_orders_customer_created;
```

The backward-compatible column rename pattern

Renaming a column directly with `ALTER TABLE orders RENAME COLUMN buyer_email TO customer_email` is fast, a pure metadata operation on most engines. The `ALTER` isn't the dangerous part. The moment it commits, every piece of code still referencing the old column name breaks atomically, everywhere, at once, including code you don't control (see the next section). So never do a rename as a single step. Expand it into phases you can ship, verify, and roll back independently:

Phase	Schema state	Application state	Rollback if something's wrong
1. Add	New column exists, nullable	Not yet used	Drop the new column - no impact

Phase	Schema state	Application state	Rollback if something's wrong
2. Dual-write	Both columns exist	App writes to both columns on every write path; reads still from the old column	Stop dual-writing - old column is still authoritative
3. Backfill	Both columns exist	Batch job populates the new column for existing rows	Backfill is idempotent - safe to re-run or pause
4. Migrate reads	Both columns exist	App reads from the new column; old column no longer read	Revert the read change - old column is still fully populated
5. Stop dual-write	Both columns exist	App writes only to the new column	Revert to dual-writing if any consumer still expects the old column
6. Drop	Old column removed	Fully migrated	Not reversible past this point - this phase only ships once nothing reads the old column

Each phase is a separate, independently deployable change. If phase 4 turns up a bug, say the new column carries a subtly different meaning than everyone assumed, the rollback is "revert one deploy" instead of "recover from a completed rename." It's the same expand-and-contract shape the shared-package upgrades in Chapter 7 use, and it generalizes to any change where old and new have to coexist for a transition window.

Phase 1 is the only phase that touches the database schema directly, and it's a plain, additive migration:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        // after() is MySQL-only; omit it on PostgreSQL.
        $table->string(
            'customer_email'
        )->nullable()->after('buyer_email');
    });
}
```

Phase 2's dual-write lives in the model, not in a migration. Until phase 5 removes it, every write path depends on that one line of application code, and "every write path" here means every write path that goes through a model instance. `Order::query()->update( ... )` and anything built on `DB::table('orders')` skip model events entirely, so grep for those before you trust this hook. Phase 4's correctness rests on it:

```
protected static function booted(): void
{
    static::saving(function (Order $order) {
        if ($order->isDirty('buyer_email')) {
            $order->customer_email = $order->buyer_email;
        }
    });
}
```

Phase 6's drop is deliberately its own migration, shipped only after the checklist at the end of this chapter is fully checked off:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        $table->dropColumn('buyer_email');
    });
}
```

Batching large backfills

Phase 3, populating the new column for two hundred million existing rows, is its own hazard if you do it in one statement:

```
// Don't do this on a large table.
DB::table('orders')
    ->update(['customer_email' => DB::raw('buyer_email')]);
```

A single `UPDATE` touching every row holds row locks, and depending on

engine and isolation level it can hold them for the whole of one very long transaction, competing with live traffic for the same rows. On replicated setups it also creates significant replication lag, because a replica applies that enormous write as one unit. Chunk it instead. Touch a bounded number of rows per statement, with a small pause between batches to let replicas catch up and to leave room for live traffic.

```
Order::query()
    ->whereNull('customer_email')
    ->orderBy('id')
    ->chunkById(1000, function ($orders) {
        Order::query()
            ->whereIn('id', $orders->pluck('id'))
            ->update(
                ['customer_email' => DB::raw('buyer_email')]
            );

        // 100ms - bound replication lag and lock contention
        usleep(100_000);
    });
```

Better still, run it as a resumable Artisan command that processes one bounded batch and records the last processed ID, instead of as a migration step. Then you can pause it, watch it, and restart it without redoing completed work, which matters for a backfill that might run for hours against a genuinely large table:

```
final class BackfillCustomerEmail extends Command
{
    protected $signature =
        'orders:backfill-customer-email'
```

```
            .' {--fresh} {--dry-run} {--batch=1000}';

    public function handle(): int
    {
        $lastId = $this->option(
            'fresh'
        ) ? 0 : (int) DB::table('backfill_progress')
            ->where('name', 'customer_email')
            ->value('last_id');

        $dryRun = $this->option('dry-run');
        $orders = Order::query()
            ->whereNull('customer_email')
            ->where('id', '>', $lastId)
            ->orderBy('id')
            ->limit((int) $this->option('batch'))
            ->get(['id']);

        if ($orders->isEmpty()) {
            $this->components->info(
                'Backfill complete: no null emails left.'
            );

            return self::SUCCESS;
        }

        $nextLastId = $orders->last()->id;

        if (! $dryRun) {
            /*
             * whereNull again, so a concurrent write that
already
             * populated one of these rows isn't overwritten by a
             * stale read.
             */
```

```
            Order::query()
                ->whereIn('id', $orders->pluck('id'))
                ->whereNull('customer_email')
                ->update(
                    ['customer_email' => DB::raw('buyer_email')]
                );

            DB::table('backfill_progress')->updateOrInsert(
                ['name' => 'customer_email'],
                ['last_id' => $nextLastId],
            );
        }

        $this->components->info(
            ($dryRun ? '[dry run] ' : '')
            ."Processed {$orders->count()} rows"
            ." (last ID: {$nextLastId})."
        );

        return self::SUCCESS;
    }
}
```

Dispatched via `Schedule::command('orders:backfill-customer-email')->everyMinute()->withoutOverlapping()`, this makes progress in small, bounded increments and survives being interrupted by a deploy at any point. The cursor lives in a table rather than in the cache for a boring reason that bites people anyway: a cache is allowed to lose your key. An eviction under `maxmemory`, a `cache:clear` during a release, or a Redis restart without persistence silently resets the backfill to ID 0. The `whereNull` guard then means it quietly grinds back through two hundred million already-completed rows without ever reporting that anything went wrong.

Give every backfill command a `--dry-run` flag before it ever touches a large table. It walks the same query and batch-selection logic, reports how many rows and which ID range the next batch would affect, and writes nothing, so a mistake in the `WHERE` clause shows up as a suspicious row count instead of two hundred million incorrectly updated rows. Print a progress line per batch too, not one message at the end. A backfill that runs for hours in silence looks exactly like one that has hung, and a per-batch line carrying the row count and the last ID reached answers "is this still running?" from the terminal.

Downstream consumers: your schema is a public API

The phased rename protects the application. It does nothing for anything else reading the same database directly outside the deploy you control: a nightly analytics export, another internal service with direct read access to the same tables (an anti-pattern, but a commonly inherited one), or a BI tool querying a read replica. From where they sit, dropping `buyer_email` in phase 6 is a breaking change shipped with zero notice, because your migration process has no idea they exist.

The fix starts with an inventory. Before any schema change on a table of nontrivial age, find out who reads it besides the application: export jobs, other services, reporting queries, scheduled dashboards. Karbar's inventory turned up an internal reporting export running nightly against `orders.buyer_email` directly, owned by a different team on a different release cycle.

Two patterns handle this without forcing every downstream consumer onto the application's timeline:

- **A compatibility view.** Keep a database view that presents the old shape, or during the dual-write window, the old column itself. A

consumer's `SELECT` list can go on naming `buyer_email` long after the table stopped having that column. They do still have to repoint the `FROM` clause at the view, a one-line change on their side rather than a rewrite, and it has to happen while the old column still exists, not on the day it disappears. Drop the view once every known consumer has confirmed it's no longer needed, on the slowest consumer's schedule rather than the application's.

- **A deprecation window with an actual notification.** Whoever owns the schema change communicates the timeline to known downstream owners the same way a public API deprecates an endpoint: an announced window instead of a silent drop. This only works if the inventory step actually happened, because a consumer nobody knew about can't be notified.

Both of those patterns are damage control for consumers that already read columns directly. For an HTTP client, the more durable fix is to never let that exposure exist. Don't return `Model::toJson()` or a bare `$order->toArray()` from a controller, and don't let a resource's field names default to mirroring column names. A Laravel API Resource sits between the two and makes the wire format an explicit, independently versioned contract. A `league/fractal` transformer does the same job on a project that predates Resources or doesn't use them:

```
final class OrderResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            // Wire format is frozen; the column behind it is
not.
            'buyer_email' => $this->customer_email,
            'status' => $this->status,
            'total_cents' => $this->total_cents,
        ];
    }
}
```

With that boundary in place, the whole six-phase rename can happen without a single HTTP API consumer noticing. The resource is the only place that has to change, and it changes the other way round from what you'd expect: it maps the new column onto the response field the API already promised, instead of the API dictating what the column is allowed to be called.

At Karbar, the old `buyer_email` column stayed in place, populated by the dual-write step, until the reporting team confirmed their nightly export had been moved to the new column name. That happened on their schedule, and only then did phase 6 ship. Had the export needed more time than the dual-write window allowed, a compatibility view would have covered the gap. Ship that view during the deprecation window, not alongside the drop, since a view that appears on the same day the column vanishes has nobody pointed at it yet:

```
-- Ships during the deprecation window, well before phase 6's
drop, so
-- consumers have time to repoint FROM orders at
orders_legacy_shape and
-- keep buyer_email in their SELECT list afterwards.
CREATE VIEW orders_legacy_shape AS
    SELECT id, customer_id, customer_email AS buyer_email,
status, total_cents
    FROM orders;
```

A schema-change checklist

```
- [ ] Tested the specific operation (not just "a migration")
against a production-sized copy of the table
- [ ] Confirmed whether the operation takes a blocking lock on
the current engine/version
- [ ] Used a non-blocking index build (CONCURRENTLY /
ALGORITHM=INPLACE) for large tables
- [ ] Renames/type changes are split into add -> dual-write ->
backfill -> migrate reads -> drop phases
- [ ] Backfills run in bounded batches with a pause, not as one
statement
- [ ] Inventoried downstream consumers (exports, other services,
BI tools) that read this table directly
- [ ] HTTP API responses go through a Resource/transformer, not a
bare model - column renames don't reach API clients
- [ ] Downstream consumers notified or bridged with a
compatibility view before the old shape is dropped
- [ ] The "drop" phase is the last, separately reviewed step -
never bundled with the rest
```

Chapter recap

- A migration's cost scales with table size and the specific engine and operation, not with how simple the change looks in code. Always test DDL against production-sized data.
- Use non-blocking index builds (`CREATE INDEX CONCURRENTLY` on PostgreSQL, `ALGORITHM=INPLACE, LOCK=NONE` on MySQL) on any table large enough that a lock would be noticed.
- Never rename or retype a column in one step. Expand it into add, dual-write, backfill, migrate reads, stop dual-write, and drop, each independently deployable and reversible until the last phase.
- Backfill large tables in bounded, pausable batches (`chunkById` plus a

small delay), not a single sweeping `UPDATE`, to protect replication and live traffic.

- Your schema is effectively a public API the moment anything outside your own deploy reads it directly: an export, another service, a BI tool. Inventory those consumers before assuming a change is safe.
- For consumers that go through HTTP rather than the database directly, a Laravel API Resource (or `league/fractal`) makes the response shape an explicit contract instead of a mirror of column names, so a column rename never reaches an API client at all.
- Bridge downstream consumers with a compatibility view or an announced deprecation window, and let the slowest consumer's timeline decide when the old shape can actually be dropped.

Chapter 16 - Rate Limiting & Abuse Protection

2 a.m., and the login endpoint won't stop ringing

The graphs look like an attack, because it is one. Login attempts on Adda, a fictional social platform, have gone from a baseline of a few hundred per minute to over forty thousand, spread across tens of thousands of IP addresses, each one trying a handful of username/password combinations before moving on. It's credential stuffing: a botnet replaying a leaked password list from an unrelated breach, hoping a slice of users reused the same password here.

The naive fix, block anyone who fails login more than five times, makes things worse. The attack is already distributed across so many source IPs that per-IP blocking barely slows it down, while a support queue fills with real users who mistyped a password twice and are now locked out for an hour. Surviving this incident well has very little to do with having the cleverest CAPTCHA. It comes down to having decided already, in calmer times, what a rate limit protects, at what granularity, and what happens to a legitimate request caught in the blast radius.

Rate limiting is not one mechanism. It's a layered set of decisions about who you're limiting, what unit of time you're limiting them over, and what you do when the limit is hit. Those decisions are much cheaper to make before an attack than during one.

Two limiter algorithms, and why the difference matters

Laravel ships with one of the two algorithms below, and it's worth knowing which one before you assume an endpoint is protected: the `RateLimiter` facade and the `throttle` middleware both count requests in cache-based fixed windows.

Fixed window counts requests in discrete buckets (e.g. "requests this calendar minute"). It's cheap, one counter and one TTL, but it has a boundary problem: a client can send its full limit at 11:59:59 and its full limit again at 12:00:01, getting two limits' worth of traffic in two seconds.

Token bucket (and its close cousin, the sliding-window log) models a bucket that refills at a steady rate and is drained by each request. It smooths out the boundary problem because there's no reset instant; the bucket's state is continuous. It costs slightly more to compute, since you need the last-refill timestamp and not just a count, but it's a much closer match to "how many requests can this client sustainably send," which is usually the question you have.

	Fixed window	Token bucket / sliding window
Storage cost	One counter + TTL per key	Timestamp + count per key
Boundary burst	Up to 2x limit across a window edge	Bounded by bucket size, not window edge

	Fixed window	Token bucket / sliding window
Good for	Coarse, cheap limits (e.g. "1000 req/hour per API key")	Limits where burst behavior matters (login, payment attempts)
Laravel primitive	`RateLimiter::attempt()` with default cache-based windows	Custom atomic bucket in Redis or an edge-provider limiter

For most application-level throttles, the fixed window's imprecision doesn't matter. Nobody cares if a reporting endpoint allows a small burst across a minute boundary. For anything security-sensitive (login, password reset, payment attempts, OTP verification), the boundary burst is the gap an attacker will find, so a token bucket is worth the extra complexity. You will be building that yourself, in Redis or at the edge, because it isn't a mode you can switch the stock limiter into.

Choosing the key: per-user, per-IP, per-action, or a combination

A rate limit is only as good as the key it's keyed on, and this is where most naive implementations fail.

- **Per-IP** limits stop a single machine from hammering an endpoint, but are close to useless against a distributed botnet, and actively harmful behind carrier-grade NAT or corporate proxies, where thousands of

real users share one visible IP.

- **Per-user (or per-account)** limits are precise but only apply once you know who the user is. They don't help on an unauthenticated login attempt where the "user" is exactly what's being guessed.
- **Per-action** limits (e.g. "5 password reset emails per hour, regardless of who's asking") protect a shared downstream resource (an email provider, an SMS gateway) from being exhausted by any single actor.
- **Composite keys**, meaning IP plus the attempted username, or device fingerprint plus account, are usually where the real protection lives, because they let you set a tight limit on the narrow combination ("this IP trying this specific username") without punishing an entire shared IP range for one bad actor.

The credential-stuffing scenario above is why single-axis keys fail: per-IP limiting is defeated by the botnet's IP diversity, and per-username limiting alone would let the botnet grind through the account list one attempt per username, staying under any reasonable per-account threshold. The fix is layered limits on different keys simultaneously, each cheap on its own, expensive to defeat all at once.

```
/*
 * A layered login throttle: IP-wide, then IP+username, then
 * account-wide.
 */
public function attempt(Request $request): RedirectResponse
{
    $ip = $request->ip();
    $username = (string) $request->input('email');

    $limits = [
        // this IP, any account, per minute
        "login-ip:{$ip}" => 30,
```

```
        // this IP against this account
        "login-ip-user:{$ip}:{$username}" => 5,
        // this account, from anywhere
        "login-user:{$username}" => 10,
    ];

    foreach ($limits as $key => $maxAttempts) {
        if (RateLimiter::tooManyAttempts($key, $maxAttempts)) {
            $seconds = RateLimiter::availableIn($key);

            throw ValidationException::withMessages([
                'email' =>
                    "Too many attempts. Retry in {$seconds}s.",
            ])->status(429);
        }
    }

    if (! Auth::attempt($request->only('email', 'password'))) {
        foreach (array_keys($limits) as $key) {
            /*
             * The first failed attempt must create the fixed
             * window's TTL.
             */
            RateLimiter::hit($key, decaySeconds: 60);
        }

        throw ValidationException::withMessages([
            'email' =>
                'These credentials do not match our records.',
        ]);
    }

    /*
     * Successful logins do not increment any of the failure
     * counters.
```

```
        */
        return redirect()->intended('/dashboard');
    }
```

Hitting the limiter only on failed attempts, rather than on every request, is what keeps this from punishing legitimate traffic. A limiter that counts successes too will eventually lock out the user whose only crime is logging in a lot.

Bot and risk scoring: don't let a vendor outage become your outage

Many teams layer a third-party risk-scoring or bot-detection service (device fingerprinting, behavioral scoring, a managed CAPTCHA) on top of application-level rate limits. Those services help, but they introduce a dependency that can fail independently of your own infrastructure, and a naive integration turns "the risk vendor is slow" into "nobody can log in."

Decide explicitly what happens when the risk check itself fails or times out, rather than letting an exception bubble up and 500 the request. It's the same fail-open-vs-fail-closed call Chapter 26 builds into a general framework for any dependency. Applied to a risk-scoring vendor, there are two defensible postures, and the right one depends on what the endpoint protects:

Failure mode	Fail closed (block)	Fail open (allow)
Behavior on vendor timeout	Treat as high-risk, block or challenge	Treat as unscored, let the request through
Right for	High-value, low-volume actions (large withdrawals, account recovery)	High-volume, low-marginal-risk actions (viewing a page, browsing a catalog)
Failure mode if wrong	Vendor blip becomes a full outage for real users	A vendor outage becomes a temporary abuse window
Mitigation	Short timeout (100-300ms) + circuit breaker so a slow vendor degrades to "unscored" quickly	Alert loudly on sustained fail-open so someone investigates rather than it becoming permanent

For Adda's login endpoint, the pragmatic default is to apply the application-level rate limits unconditionally, since they are self-hosted and always available, and to treat the risk score as an additional signal that can tighten the response, for example by requiring a CAPTCHA. That score should not be a hard dependency for the base throttle to function. A risk-vendor outage then degrades the system to "slightly less sophisticated bot detection," not "login is down."

In code, that means a tight timeout on the vendor call and an explicit fail-open branch, rather than letting the exception propagate:

```
public function scoreLogin(string $ip, string $email): ?RiskScore
{
    try {
        $response = $this->http
            // 250ms: fail fast, don't hold up the login request
            ->timeout(0.25)
            ->post('https://risk-vendor.example.com/v1/score', [
                'ip' => $ip,
                'email' => $email,
            ]);

        if ($response->failed()) {
            /*
             * A 4xx/5xx arrives here, not in the catch block
             * below.
             */
            Log::critical(
                'Risk vendor returned an unsuccessful response',
                [
                'status' => $response->status(),
            ]);

            return null;
        }

        return RiskScore::fromArray($response->json());
    } catch (ConnectionException $e) {
        /*
         * Fail open: an unscored login still passes the
         * application-level throttle above, it only skips the
         * extra CAPTCHA/challenge step.
         */
        Log::critical(
            'Risk vendor unavailable, failing open',
```

```
                ['exception' => $e->getMessage()]
            );

            return null; // caller treats null as "unscored"
        }
    }
```

The loud `Log::critical` call is what keeps a fail-open decision visible instead of silently permanent.

DDoS protection lives above Laravel

The credential-stuffing attack that opened this chapter is abuse: many small, well-formed requests that look enough like real logins to reach your application. A distributed denial-of-service (DDoS) attack is a different shape. It sends enough traffic, or holds enough half-open connections, to exhaust bandwidth, file descriptors, or worker capacity before your code ever gets a vote. Laravel's `RateLimiter` never fires, because the request never makes it to a PHP worker. Treating those two problems as the same "rate limit" problem is how teams spend a week tuning `throttle:60,1` while the load balancer is already dropping SYN packets.

The layer that can absorb a volumetric flood is the one in front of your origin: a CDN or DDoS-aware edge (Cloudflare, AWS Shield / WAF, Fastly, and their peers), with connection limits and SYN cookies at the load balancer as a second line. That edge is where you put IP reputation, geographic blocks you actually intend to enforce, managed bot challenges, and traffic shaping that can drop junk without waking a Laravel container. Application rate limits still matter for the abuse cases this chapter covers. They are just not the DDoS plan.

Layer	What it can stop	What it cannot
Edge / CDN / WAF	Volumetric floods, obvious botnets, TLS exhaustion, cheap challenge-before-origin	Business-logic abuse that looks like a real user (credential stuffing with residential IPs)
Load balancer / reverse proxy	Connection storms, slowloris-style holds, per-IP connection caps	Anything that already looks like a healthy HTTP request
Laravel `RateLimiter` / `throttle`	Per-key abuse once a request reaches the app	Floods that saturate the link or the worker pool before PHP runs

For Adda, the practical split after the stuffing incident was: keep the layered login throttles in the app, since they still catch the "looks like a user" case, and put volumetric protection at the edge so a packet flood never becomes a Horizon backlog. Two other habits make the origin cheaper to defend when something does leak through:

- **Fail expensive work late.** Don't run bcrypt, Eloquent, or a risk-vendor call before the cheapest checks (routing, auth middleware, a cache hit on a known-bad IP). Every millisecond of origin work during a flood is capacity an attacker can buy more cheaply than you can.
- **Keep health and static paths cheap and separate.** A `/up` or health probe that still boots the full application, or a homepage that always misses cache and hits the database, turns "protect the login endpoint"

> into "the whole fleet is busy answering probes and HTML." Edge caching for anonymous GETs, and a health check that doesn't touch Redis or MySQL, shrink the surface an attack can usefully burn.

Do not invent an in-app "DDoS mode" that flips every limit to zero when CPU is high. That couples abuse policy to capacity noise, locks out real users during a legitimate spike (the flash-sale case Chapter 17 plans for), and still does nothing if the attack never reaches PHP. Edge absorption first; application throttles for the requests that deserve application judgment.

Backpressure: what a limit does after it triggers

A rate limit does more than reject the request. What you return, and what you do internally when a limit is being approached, is its own design surface:

- **Return `429 Too Many Requests`**, not a generic `403` or `500`. Well-behaved clients and API consumers know to back off on a 429 specifically, and a `Retry-After` header lets them do it without guessing.

Concretely, that's `RateLimiter::availableIn()` feeding straight into the response:

```
public function login(Request $request): JsonResponse
{
    $key = "login-ip:{$request->ip()}";

    if (RateLimiter::tooManyAttempts($key, 30)) {
        $retryAfter = RateLimiter::availableIn($key);

        return response()->json(
            [
                'message' =>
                    "Too many attempts. Retry in {$retryAfter}s."
            ],
            429,
        )->header('Retry-After', $retryAfter);
    }

    // ...
}
```

- **Shed load progressively, not as a cliff.** If an upstream dependency (a payment gateway, a third-party API) is itself struggling, consider tightening your own limits proactively rather than waiting for it to start timing out request by request. This is the same backpressure idea covered for queues in Chapter 13, applied at the edge instead of the worker layer.
- **Distinguish abuse throttling from capacity throttling.** A login rate limit exists to stop abuse and should feel deliberately strict. A general API rate limit protecting your own database from being overwhelmed is a capacity control, and Chapter 17's load-testing work is what tells you where that number should sit. Guessing produces either a limit so loose it doesn't protect anything, or one so tight it throttles normal peak traffic.

Chapter recap

- Rate limiting is a layered decision, not a single mechanism: pick an algorithm (fixed window for cheap coarse limits, token bucket for anything where burst behavior matters), a key (IP, user, action, or a composite), and a response (reject, challenge, or degrade).
- Composite keys (IP + username, device + account) catch attacks that defeat any single-axis limit, including distributed credential stuffing that no per-IP limit alone can stop.
- Increment attempt counters on failure, not on every request, or you'll rate-limit your legitimate power users.
- Third-party risk/bot signals are an enhancement, not a foundation. Decide explicitly whether each check fails open or closed, and never let a vendor timeout become your outage.
- DDoS is an edge problem, not a `RateLimiter` problem: absorb volumetric floods at the CDN/WAF/load balancer, keep origin work cheap for what leaks through, and reserve application throttles for abuse that looks like a real request.
- Return `429` with `Retry-After`, and treat abuse throttling (strict, security-motivated) and capacity throttling (informed by load testing, see Chapter 17) as two different tuning problems with two different failure costs.

Chapter 25 - Working with Datetime and Timezones

The streak that broke at midnight

Drishti, a fictional livestreaming and short-video app, runs a seasonal campaign with a daily streak: watch or post on consecutive local days and unlock a reward on Monday. The product copy says "day" the way a user means it: midnight to midnight in Pacific time, where the company and most of its creators sit. The implementation means something else.

`APP_TIMEZONE` is `UTC`, which is correct for storage. Streak logic calls `now()->startOfDay()` and stores the result as the active date. At 11:30 p.m. Pacific on a Tuesday, that is already Wednesday UTC. A creator who posts once on Tuesday evening and once on Wednesday morning, two distinct Pacific calendar days, gets a single streak day, because both actions fall on the same UTC date. Support tickets pile up the week the campaign goes viral, and engineering "fixes" it by flipping the app timezone to `America/Vancouver`. Overnight, every `created_at` comparison, every TTL, and every cron that assumed UTC shifts by seven or eight hours depending on daylight saving, and Monday's payout job either double-fires or doesn't fire at all.

Timezone bugs are correctness bugs. They don't show up as 500s. They show up as the wrong calendar day, the wrong week boundary, a schedule that runs at the wrong wall-clock time, or an admin panel that displays yesterday as today. Keeping them out means keeping two clocks straight, **UTC for instants** and a named **business timezone for calendar math**, along with the edge cases that appear the moment a product says "day," "week," or

"Monday at midnight."

Instants and calendar days are different types

An *instant* is a point on the timeline: "this ticket was created at this exact moment," stored once, comparable everywhere. A *calendar day* is a civil label that depends on a timezone, so "Tuesday, March 10" in Vancouver is not the same set of instants as "Tuesday, March 10" in UTC. Mixing the two is the root failure mode.

Question	Use
When did this row get written?	UTC instant (`timestampTz`, `now()` in UTC)
Is this user still inside a 24-hour redeem window?	Duration from a UTC instant (`subHours(24)`)
Did they act on consecutive local days?	Convert to business TZ, then `toDateString()`
Does "this week" mean Mon-Sun for the campaign?	Week boundaries in the business TZ
When should the Monday payout cron run?	Scheduler `->timezone( ... )` matching that calendar

Laravel's `config('app.timezone')` should almost always be `UTC`. That makes `now()`, Eloquent timestamps, and queue `available_at` values mean the same thing on every server and in every region. Business calendar

semantics belong in application code as an explicit constant, not in `APP_TIMEZONE`, and not as a per-row column unless you genuinely need per-user timezones (wallets with local spending days, Chapter 23's `spendingDay`, are that case; a single-company campaign usually is not).

```
final class CampaignConfig
{
    /*
     * Calendar days, weeks, and payout schedules for this
     * product. DB timestamps stay UTC instants; convert to
     * this zone only when asking civil-date questions.
     */
    public const BUSINESS_TIMEZONE = 'America/Vancouver';
}
```

One constant beats a `timezone` column you will forget to set on half the rows, and beats three competing literals (`America/Vancouver`, `America/Los_Angeles`, `PST`) scattered across jobs.

Convert at the boundary, not in the middle

When an event arrives, whether a watch, a purchase, or a ticket grant, keep the instant in UTC and derive the civil day only where the product rule needs it:

```
public function recordActiveDay(
    int $userId,
    CarbonImmutable $activeAt,
): void {
    $activeDate = $activeAt
        ->timezone(CampaignConfig::BUSINESS_TIMEZONE)
        ->startOfDay();
    $today = $activeDate->toDateString();

    $progress = StreakProgress::query()
        ->lockForUpdate()
        ->firstOrCreate(
            [
                'user_id' => $userId,
                'active_date' => $today,
            ],
        );

    /*
     * ... streak increment using $today / yesterday
     * in the same business timezone
     */
}
```

The move that matters is `->timezone(BUSINESS_TIMEZONE)` before `startOfDay()` or `toDateString()`. Calling `startOfDay()` in UTC and then "displaying" Pacific is how you mint the wrong day. The inverse shows up in queries. When you need "everything since one week ago at Pacific midnight," compute that bound in the business zone and convert back to UTC for the SQL comparison:

```
$weekAgoPacificMidnightUtc = CarbonImmutable::now(
    CampaignConfig::BUSINESS_TIMEZONE,
)
    ->subWeek()
    ->startOfDay()
    ->utc();

Campaign::query()
    ->where('ends_at', '>=', $weekAgoPacificMidnightUtc)
    ->get();
```

That pattern, civil math in the business zone and storage and filters in UTC, is the same idea Chapter 23 uses when it freezes `spendingDay` at decision time so replay never re-asks "what is today?"

Schedulers must declare the timezone they mean

Laravel's scheduler inherits `app.timezone`. If that is UTC, then `->dailyAt('00:05')` means 00:05 UTC, which is not "just after midnight for the Pacific campaign." In practice this fails more quietly than a wrong app timezone: one job uses `->timezone('America/Vancouver')`, a sibling job forgets and runs in UTC, and both claim to be "the Monday payout."

```
$schedule->job(new DisburseStreakRewards)
    ->weekly()
    ->mondays()
    ->at('00:05')
    ->timezone(CampaignConfig::BUSINESS_TIMEZONE)
    ->withoutOverlapping()
    ->onOneServer();

$schedule->command('campaign:sync-all-items')
    ->dailyAt('06:00')
    ->timezone('UTC')
    ->withoutOverlapping()
    ->onOneServer();
```

Be explicit even when the answer is UTC. A codebase that mixes Vancouver payouts, New York compliance jobs, and Los Angeles digests works fine, as long as each entry names its zone next to the clock time and that zone matches the product rule the job implements. A schedule entry with no `->timezone()` is not timezone-agnostic. It means "whatever `app.timezone` is today," which is the wrong default for wall-clock work.

Uniqueness keys must use the same clock as the schedule

`ShouldBeUnique` and cache locks are timezone-sensitive when the key embeds a date or hour. A streak reward job that must run once per Pacific calendar day needs a Pacific day in `uniqueId()`. A missing-ticket generator that may safely run once per UTC hour needs a UTC hour. Using the wrong one either blocks a legitimate second run or allows a duplicate:

```
public function uniqueId(): string
{
    /*
     * Pacific calendar day - aligned with the Monday
     * Vancouver schedule, not with UTC midnight.
     */
    return $this->campaign->slug.'::'.now(
        CampaignConfig::BUSINESS_TIMEZONE,
    )->format('Y-m-d');
}
```

```
public function uniqueId(): string
{
    // Hourly reconciliation: UTC hour is the right grain.
    return $this->campaign->slug.'::'.now()
        ->format('Y-m-d-H');
}
```

If the unique key and the schedule disagree about which midnight matters, you get the incident the unique constraint was supposed to prevent, either skipped work or double payouts, and it reproduces only near timezone boundaries.

"Day" sometimes means 24 hours

Not every product "day" is a calendar day. A redeem cap of "once per day" often means "no more than once in any rolling 24-hour window," which is a duration from a UTC instant:

```
$recent = Redeem::query()
    ->where('user_id', $userId)
    ->where('created_at', '>=', now()->subHours(24))
    ->exists();
```

A weekly purchase limit that uses `startOfWeek()` in UTC while marketing promises "this Pacific week" is a quieter cousin of the streak bug. Decide deliberately:

- **Rolling duration** (`subHours(24)`, `subDays(7)`) when the rule is elapsed time.
- **Civil calendar** (business-TZ `startOfDay` / `startOfWeek`) when the rule is a named day or week the user would recognize on a wall calendar.

Write the choice down next to the constant. The incident is almost always someone using the other interpretation without noticing.

Don't write time as a magic number

Durations have the same readability problem as timezones. A bare `300` in a cache call does not say whether the author meant five minutes, and once Laravel started treating integer TTLs as seconds, half a codebase's "I thought that was minutes" assumptions turned into silent bugs. Use a helper that names the unit.

```
use function Illuminate\Support\{
    minutes,
    seconds,
    hours,
    days,
};

/*
 * Readable at the call site - five minutes, not "300 of
 * something." These return a CarbonInterval the cache
 * layer accepts directly.
 */
Cache::put('trending', $feed, minutes(5));
Cache::remember(
    'exchange-rate',
    seconds(30),
    fn () => $this->fetchRate(),
);
Cache::put('sitemap', $xml, hours(6));
Cache::put('annual-report', $pdf, days(1));
```

`now()->addMinutes(5)` (or `now()->plus(minutes: 5)`) is the same idea when you need an absolute expiry instead of an interval, and Chapter 11's cache examples use that form. What to avoid is the unadorned integer and the arithmetic that pretends to document itself:

```
// Opaque: seconds? minutes? a typo for 30?
Cache::put('trending', $feed, 300);

// Still a magic number, just wearing a disguise.
Cache::put('trending', $feed, 5 * 60);
```

If a TTL is a product decision reused in more than one place, lift it to a named constant (`private const TRENDING_TTL = ...` backed by `minutes(5)`), the same way Chapter 19 names `TTL_MINUTES` next to the partner idempotency cache. The goal is that a reviewer can read the call site and know the unit without doing mental arithmetic or checking the framework docs for which epoch the integer means this year.

Admin panels lie in the default timezone

Operators debug production in the admin UI. If the panel renders `created_at` in UTC while everyone on the team thinks in Pacific, someone approves the wrong day's payout, reopens the wrong dispute, or swears a job "didn't run Monday" because the timestamp looks like Sunday evening.

Set a display default in the business timezone, and make sure date columns actually go through the datetime formatter, since a bare text column ignores the panel timezone entirely:

```
/*
 * Panel default: America/Vancouver. Columns still need
 * ->dateTime() (or equivalent) so the timezone applies;
 * a plain TextColumn shows the raw UTC string.
 */
Table::configureUsing(function (Table $table): void {
    $table->timezone(
        CampaignConfig::BUSINESS_TIMEZONE,
    );
});
```

Label ambiguous fields in the UI ("Starts at (Pacific)") when compliance or finance will read them. For user-facing legal timestamps where UTC is the contract, label UTC explicitly instead of "helpfully" converting.

DST, ambiguous hours, and tests that freeze the wrong clock

`America/Vancouver` (and any IANA zone with daylight saving) has days that are 23 or 25 hours long. Local times between 2:00 and 3:00 may not exist, or may exist twice, on transition nights. Prefer:

- IANA names (`America/Vancouver`), never fixed offsets (`UTC-8`) or abbreviations (`PST` / `PDT`) as the source of truth. A fixed offset is wrong half the year wherever daylight saving applies. `UTC-8` is Pacific Standard Time, so every civil midnight you compute with it during PDT is an hour off. Abbreviations are worse: they name a single offset (`PST` vs `PDT`), they collide across regions (`CST` is Central in North America and China Standard Time elsewhere), and they force every call site to know which seasonal label is "current" instead of letting the

zone database apply the transition rules.
- Civil boundaries via `startOfDay()` / `startOfWeek()` in that zone, not `setTime(0, 0)` on a UTC instant you hope is midnight.
- Storing the result of calendar decisions when they must survive replay (Chapter 23's frozen `spendingDay`), instead of recomputing "today" later.

Tests are where these bugs either get caught or get permanently skipped. Pin time in the zone the rule uses:

```
public function test_streak_counts_pacific_days(): void
{
    CarbonImmutable::setTestNow(
        CarbonImmutable::parse(
            '2026-03-10 23:30:00',
            CampaignConfig::BUSINESS_TIMEZONE,
        ),
    );

    /*
     * act in Pacific evening...
     * assert active_date === '2026-03-10', not 03-11
     */
}
```

Also test the UTC side of the same instant (`06:30` the next day UTC during PDT) so a future refactor that drops the `->timezone( ... )` call fails loudly. Boundary cases worth a dedicated test: one second before and after Pacific midnight, the spring-forward gap, the fall-back overlap, and the first/last partial week of a campaign that does not start on Monday.

A short checklist before shipping calendar logic

1. `app.timezone` is UTC; business calendar TZ is a named constant.
2. Every `startOfDay` / `startOfWeek` / `toDateString` for product rules runs after converting to that constant.
3. Every scheduler entry that cares about wall-clock time calls `->timezone( ... )` explicitly.
4. Unique locks and idempotency keys embed dates or hours in the same zone as the schedule or product rule they protect.
5. Rolling windows use durations, civil windows use the business zone, and never both for the same rule.
6. Cache TTLs and other durations use `minutes()` / `seconds()` / `hours()` / `days()` (or `now()->addMinutes( ... )`), not bare integers like `300`.
7. Admin date columns format through the panel timezone; labels say which zone the human is looking at.
8. Tests freeze time in the business zone and assert the civil date, not only the UTC timestamp.

Chapter recap

- Timezone bugs are silent correctness failures: wrong streak days, wrong week boundaries, wrong cron walls, wrong admin timestamps, usually without an exception.
- Keep `app.timezone` at UTC for instants; put civil-calendar semantics in an explicit business-timezone constant and convert only at the boundary where the product asks a calendar question.
- Schedulers and uniqueness keys must name the same timezone as the product rule they implement; "default app timezone" is not a safe stand-in for "Monday midnight Pacific."

- Decide whether "day" means a rolling duration or a civil calendar day, and encode that choice once. Mixing the two is how redeem caps and streak counters drift apart.
- Write durations with unit-named helpers (`minutes(5)`, `seconds(30)`, `now()->addMinutes(5)`), not magic numbers. Bare `300` hides the unit and has already bitten teams when cache TTLs changed from minutes to seconds.
- Admin UIs need the business timezone and actual datetime formatting; otherwise operators debug the wrong clock.
- Prefer IANA zones, freeze civil decisions that must survive replay, and write tests around Pacific midnight and DST transitions. Those are the hours production will find if you don't.

This is a sample from "Laravel After Deploy: Architecture, Performance, and Operations at Scale".